

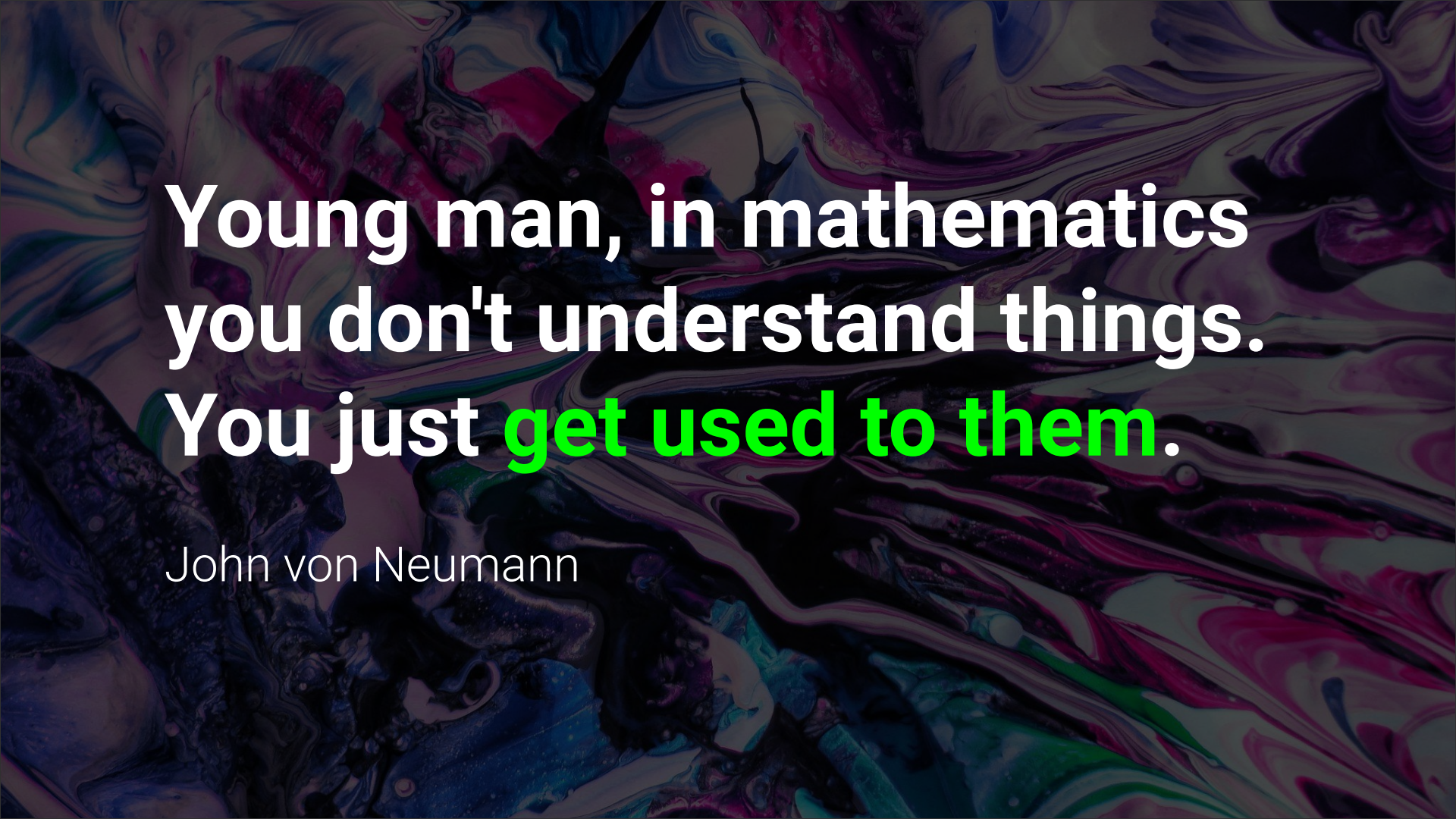
RECURSION

By Hithesh Vurjana

RECURSION

By Hithesh Vurjana





Young man, in mathematics
you don't understand things.
You just **get used to them.**

John von Neumann

What is the problem?

1. How to repeat in a **simpler way**?
2. How to repeat with **less code**?
3. How to repeat **without loops**?
4. Is there an alternative to **iteration**?
5. Is there an easier way to traverse **hierarchical structures**?



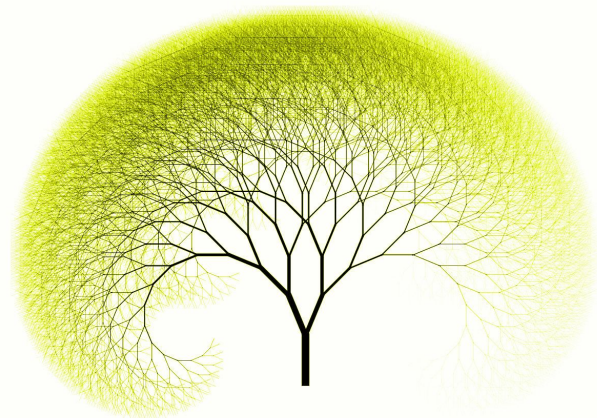
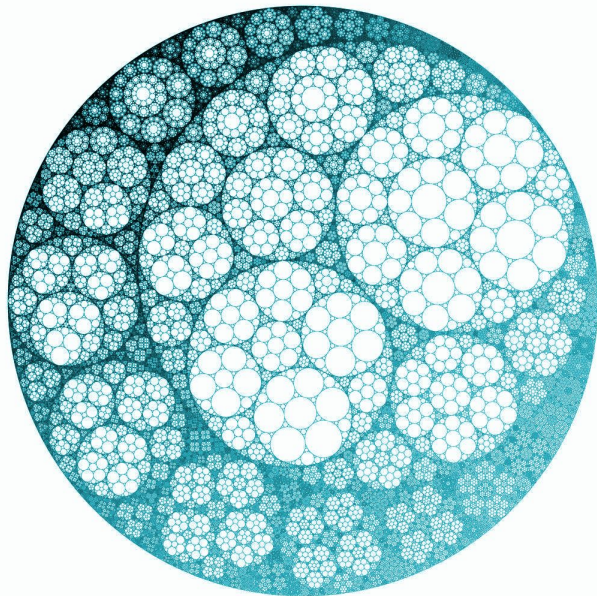
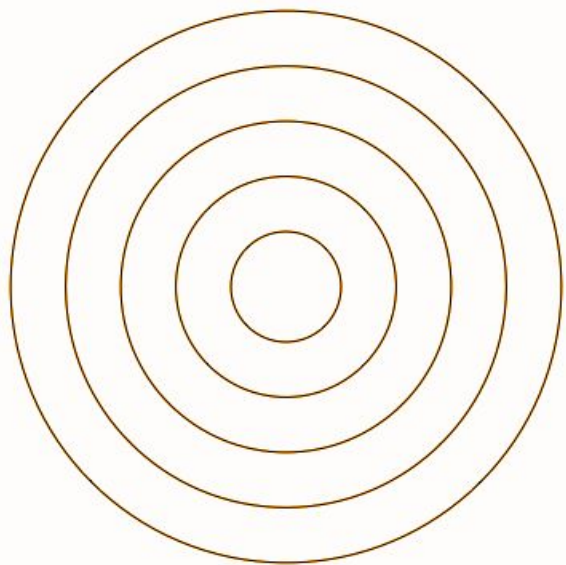
Code Time: Recursion as a solution

```
fun myFunction() {  
    myFunction()  
}
```

What is Recursion?

1. Something defined in terms of **itself**
2. Example: $2 = (2 * 2) / 2$
3. Analogy: Its like iteration **without loops**
4. Programming paradigm & problem solving **technique**
5. Technical definition: Function calls **itself**





Recursion vs Iteration

1. Iteration is repetition using **loops** while recursion is repetition when **function calls itself**.
2. Repetition stops in recursion with **base case**.
3. Repetition stops in iteration when **repetition condition** is not satisfied.
4. Any recursive solution can be implemented as an iterative solution with a **stack**.
5. **Tail Recursion** is used by compilers to convert recursive stuff to iterative stuff for optimisation.



Why Recursion?

1. As simple as a **function call**.
2. No need of **loop** structures.
3. Takes **less time** to write, maintain & debug.
4. Generally **elegant, easy** to read & understand.
5. Often **less code** than iterative approaches.



Why not Recursion?

1. **No base case** == Infinite repetitions == Stack overflow
2. **No caching** or memoization == Long running times
3. Function calls (Call Stack Ops) == **Expensive** compared to loops
4. Generally, Recursion == **Exponential** run time while iteration == Linear run time



When to use Recursion?

1. When you want to write **less code** and time complexity is not an issue (less repetitions)
2. When you are dealing with **hierarchical** stuff like tree data structures
3. Use recursion over iteration for problems that can be broken down into **multiple, smaller, independent, similar** problems.
4. Iteration is better for program. Recursion is better for programmer. **Credit**



Time & Space Complexity

1. It's about **efficiency** of algorithm
2. Rate of growth of time/space w.r.t input
3. Assume all are using same spec computer
4. A 20TB file sent digitally vs physically
5. Digital transfer $O(N)$ - Linear time
6. Physical transfer $O(1)$ - Constant time



Asymptotic Analysis

1. Big O - Worst case
2. Big Theta - Average case
3. Big Omega - Best case



Common Big O Runtimes

1. $O(1)$ - Constant Time
2. $O(\log N)$ - Logarithmic time
3. $O(N)$ - Linear time (N is input size)
4. $O(N^2)$ - Quadratic time
5. $O(N^c)$ - Polynomial time (c is constant)
6. $O(c^N)$ - Exponential time
7. $O(N!)$ - Factorial time



Basic Terminology

1. Recursion
2. Iteration
3. Base Case
4. Call Stack
5. Call / Stack Frame
6. Stack Overflow
7. Dynamic Programming

money

deficit

define /

actly w

भाव को

How to identify?

1. Design an algo to **compute the nth...**
2. Write code to **list the first n...**
3. Implement a method to **compute all...**



Structure

1. Function has **return value**
2. Function has **base condition**
3. Function has **recursive procedure**



Code Time: Sum of all values from 1 to n (Iterative)

```
fun addIteratively(n: Int): Int {  
    var result = 0  
    repeat(times: n + 1) { it: Int ->  
        result += it  
    }  
    return result  
}
```

Code Time: Sum of all values from 1 to n (Recursive)

```
fun addRecursively(n: Int): Int {  
    if (n <= 1) return 1  
    return n + addRecursively(n - 1)  
}
```

Return Value

Base condition

Recursive Procedure

- $\text{addRecursively}(1) = 1$ (This is a subproblem)
- $\text{addRecursively}(2) = 2 + 1 = 2 + \text{addRecursively}(1)$
- $\text{addRecursively}(3) = 3 + 2 + 1 = 3 + \text{addRecursively}(2)$
- $\text{addRecursively}(n) = n + (n - 1) + (n - 2) \dots = n + \text{addRecursively}(n - 1)$

Types of Recursion

1. **Direct** Recursion
2. **Indirect** Recursion
3. **Tail** Recursion
4. **Non-Tail** Recursion



Code Time: Direct Recursion

```
fun factorial(n: Int): Int {  
    return if (n == 0) 1 else n * factorial(n - 1)  
}
```

- Calls same function

Code Time: Indirect Recursion

```
fun odd(n: Int): Int = if (n < 10) even(n + 1) else 0
fun even(n: Int): Int = if (n < 10) odd(n + 1) else 0
```

- func1 calls func2 calls func1
- Write a program to print numbers from 1 to 10 in such a way that when number is odd, add 1 and when number is even, subtract 1.
- Output: 2 1 4 3 6 5 8 7 10 9

Code Time: Tail Recursion

```
fun myFunc(n: Int) {  
    if (n == 0) return else print(n)  
    return myFunc(n - 1)  
}
```

- a.k.a Tail Call Optimisation
- Means recursive call is the last evaluation done by the function
- No need to keep record of the previous state
- Output: 3 2 1

Code Time: Non-Tail Recursion

```
fun myFunc(n: Int) {  
    if (n == 0) return  
    myFunc(n - 1)  
    print(n)  
}
```

```
fun myFunc(n: Int): Int {  
    return if (n == 1) 0  
    else 1 + myFunc(n / 2)  
}
```

- Means recursive call is NOT the last evaluation done by the function
- After return there are things left for evaluation
- Output: 1 2 3

Stack Data Structure

1. Linear Data Structure
2. LIFO (Last In, First Out)
3. `push(item: Any) -> Insertion  $O(1)$`
4. `pop() -> Deletion  $O(1)$`
5. `peek() -> Access/Search  $O(n)$`

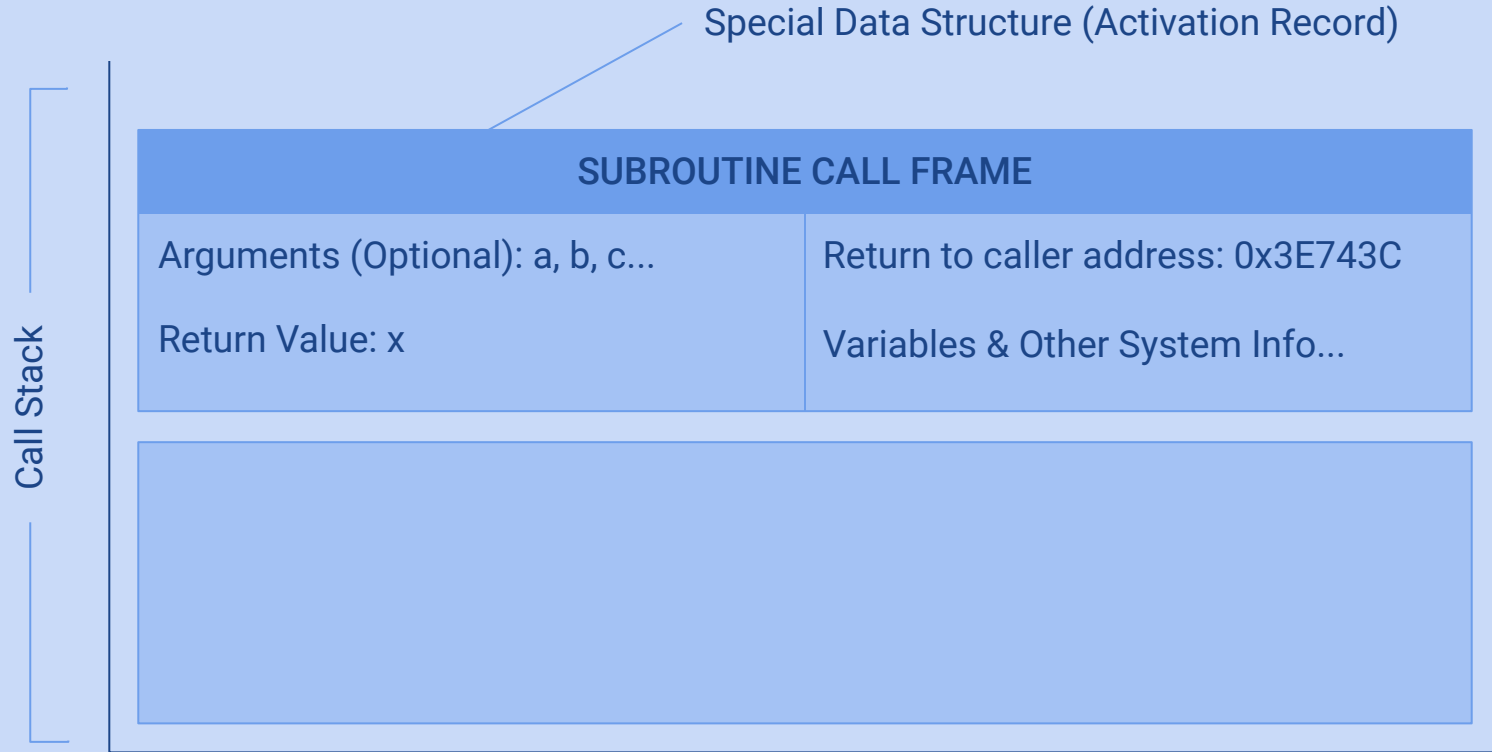


Call Stack

1. Stack **Data Structure** with **fixed** memory
2. Keeps **track** of function calls - Current running func, func within func, etc.
3. Func call -> Compiler adds func to Call Stack
4. Func done -> Compiler removes func from Call Stack
5. Empty Call Stack -> All funcs executed
6. Call Stack Memory exceeded -> **Stack Overflow**



Call / Stack Frame



Code Time: Call Stack

```
fun f1() = "My ${f2()}"  
fun f2() = "name ${f3()}"  
fun f3() = "is ${f4()}"  
fun f4() = "Hithesh"
```

f4 = "Hithesh"

f3 = "is \${f4()}"

f2 = "name \${f3()}"

f1 = "My \${f2()}"

f3 = "is Hithesh"

f2 = "name is Hithesh"

f1 = "My name is Hithesh"

Code Time: Stack Trace

```
fun main() { f1() }  
fun f1() = "My ${f2()}"  
fun f2() = "name ${f3()}"  
fun f3() = "is ${f4()}"  
fun f4(): String {  
    Exception("Stack Trace")  
        .printStackTrace(System.out)  
    return "Hithesh"  
}
```

DEBUG CONSOLE

Filter

```
java.lang.Exception: Stack Trace  
    at MainKt.f4(main.kt:123)  
    at MainKt.f3(main.kt:121)  
    at MainKt.f2(main.kt:120)  
    at MainKt.f1(main.kt:119)  
    at MainKt.main(main.kt:118)  
    at MainKt.main(main.kt)
```

Code Time: Stack Overflow

```
fun myFunction() {  
    myFunction()  
}
```

Exception in thread "main" java.lang.StackOverflowError
at MainKt.myFunction([main.kt:81](#))
at MainKt.myFunction([main.kt:81](#))

- Missing base case $\Rightarrow$ Call Stack out of memory $\Rightarrow$ Stack Overflow Error

myFunction()

myFunction()

myFunction()

myFunction()

myFunction()

Code Time: Fibonacci Series

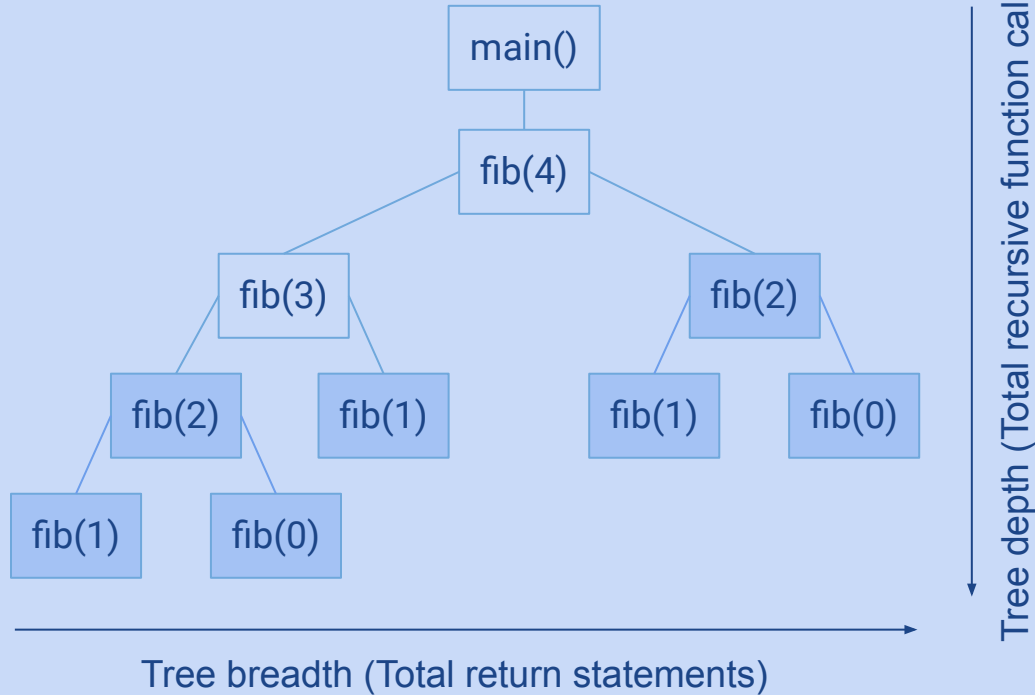
- Compute Nth term of Fibonacci Series (OR) Calculate the sum of all Fibonacci numbers of a given number.
- Fibonacci Series is a sequence of integers where the next integer in the series is the sum of the previous two starting from 0 and 1.
- 0, 1, 1, 2, 3, 5, 8, 13... where 8 is a sum of 5 and 3, its preceding 2 numbers.
- Recursive formula: $F(0) = 0$, $F(1) = 1$, $F(N) = F(N - 1) + F(N - 2)$

Code Time: Nth term of Fibonacci Series

```
fun fibonacci(n: Int): Int {  
    if (n == 0 || n == 1) return n  
    return fibonacci(n - 1) + fibonacci(n - 2)  
}
```

- $\text{fibonacci}(0) = 0$
- $\text{fibonacci}(1) = 1$
- $\text{fibonacci}(2) = 1 + 0 = \text{fibonacci}(1) + \text{fibonacci}(0)$
- $\text{fibonacci}(n) = (n - 1) + (n - 2) \dots = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$

Fibonacci Recursion Tree



Code Time: Big O of Iterative Fibonacci

```
fun fibonacci(n: Int): Int {  
    if (n == 0 || n == 1) return n  
    var prevFib = 1; var fib = 1  
    for (i in 2 until n) {  
        val temp = fib  
        fib += prevFib  
        prevFib = temp  
    }  
    return fib  
}
```

- Time Complexity: $O(N)$ - For loop with size N .
- Space Complexity: $O(N)$ - No additional data structures used other than 3 variables and the input parameter of size N .

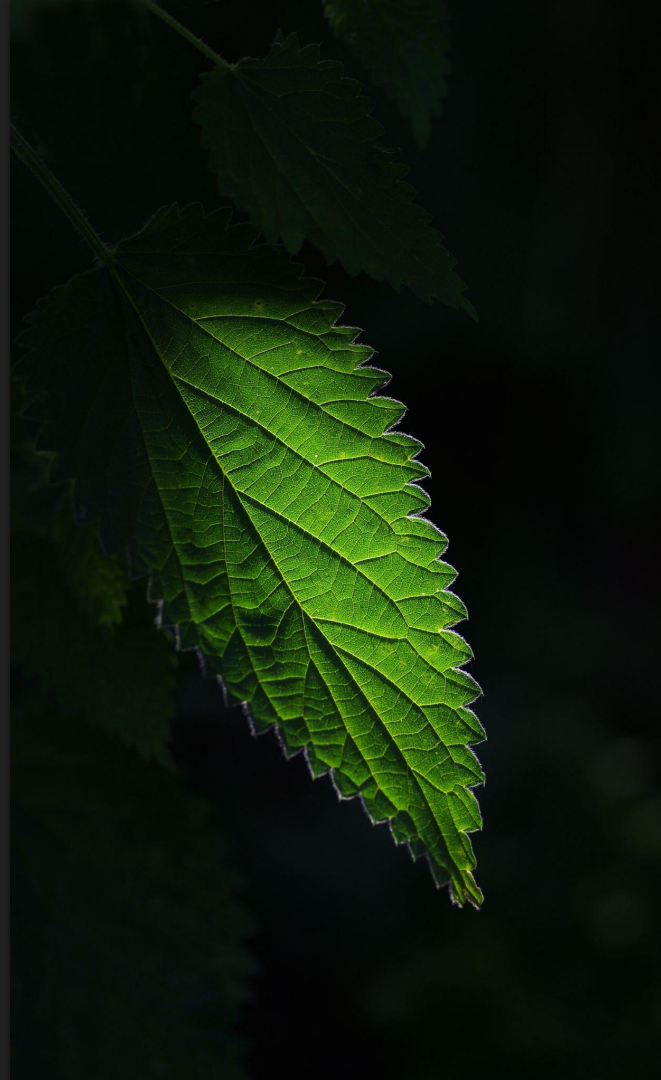
Code Time: Big O of Recursive Fibonacci

```
fun fibonacci(n: Int): Int {  
    if (n == 0 || n == 1) return n  
    return fibonacci(n - 1) + fibonacci(n - 2)  
}
```

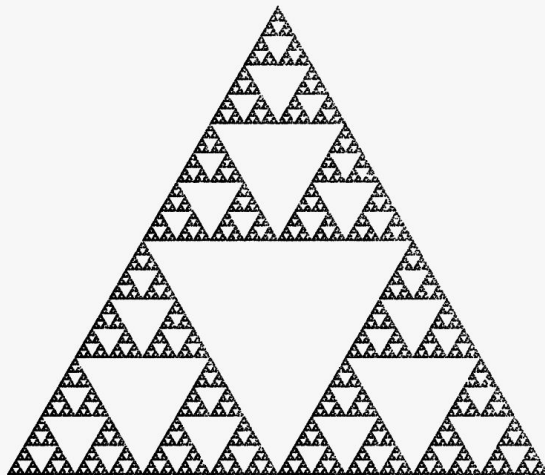
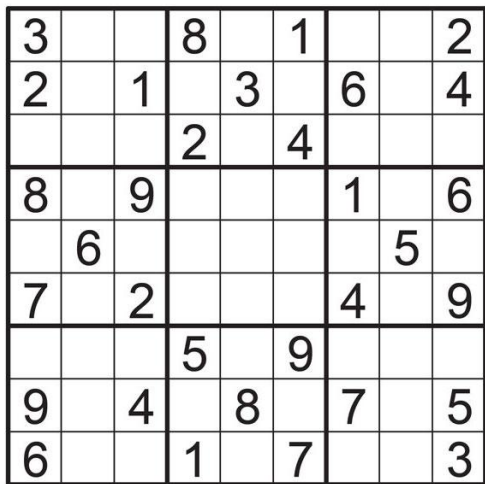
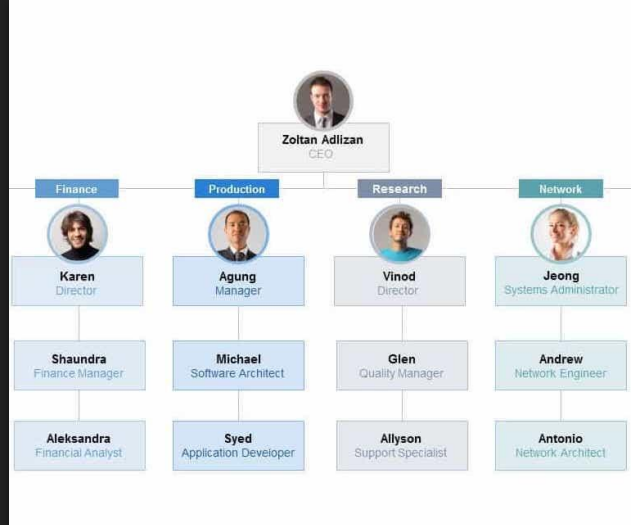
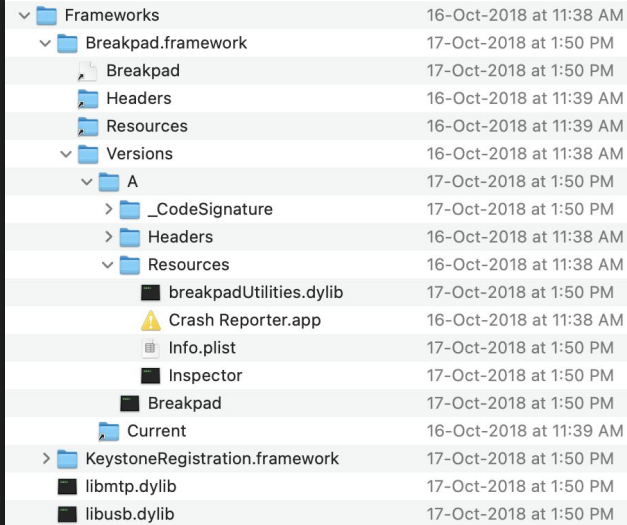
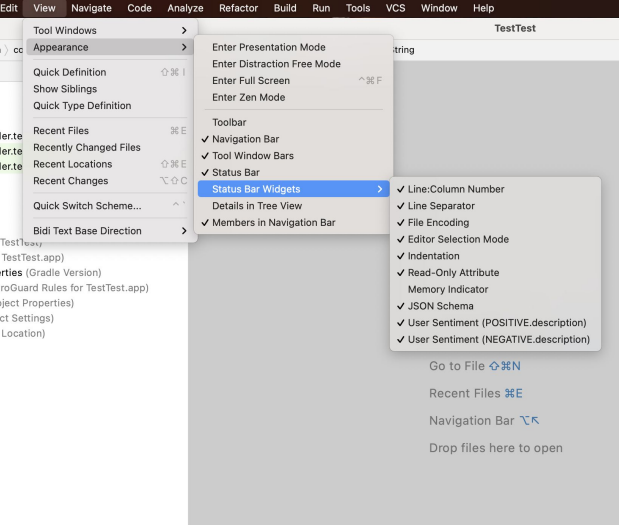
- Time Complexity: $O(2^N)$ - Each function call calls the same function twice.
- Space Complexity: $O(N)$ - Length of fibonacci calls in the call stack.

Recursion in the world

1. File directories, Menus...
2. Organisation structure
3. Game Solvers - Ex: Sudoku
4. Fractal Designs
5. Inductive Proofs
6. All Hierarchies
7. The way humans understand stuff

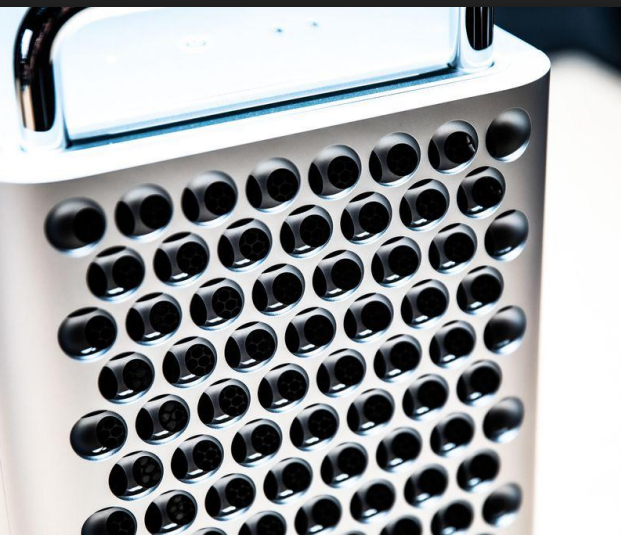
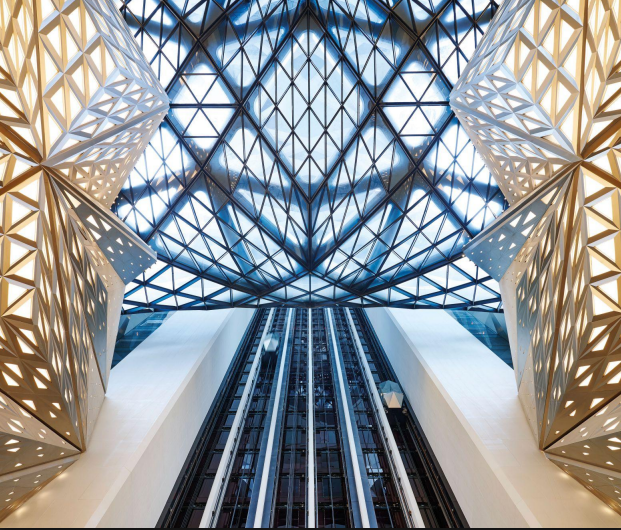












“I love you,” said Bekka.

“I know,” I said.

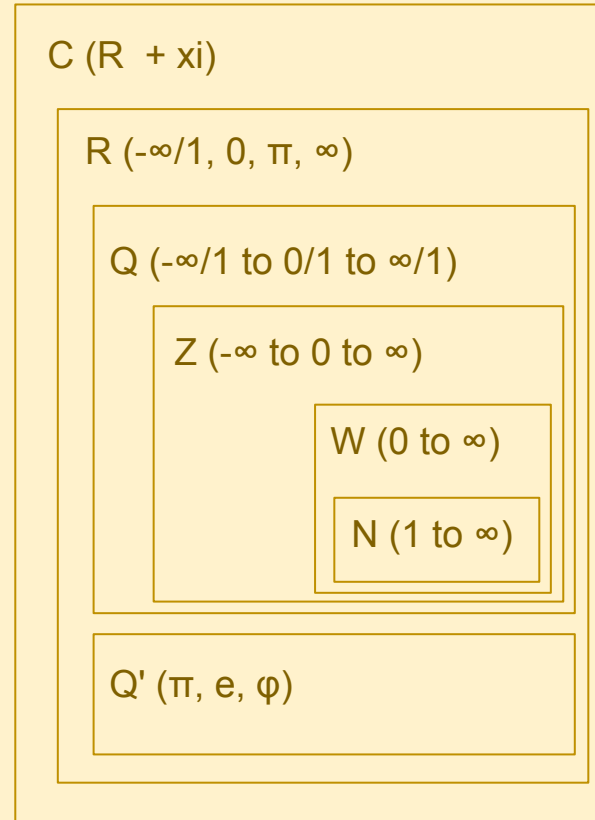
“I know you know,” she said. “But I didn’t know that you knew I knew you knew. And now I do.”

Scott Alexander, *It Was You Who Made My Blue Eyes Blue*

Number System

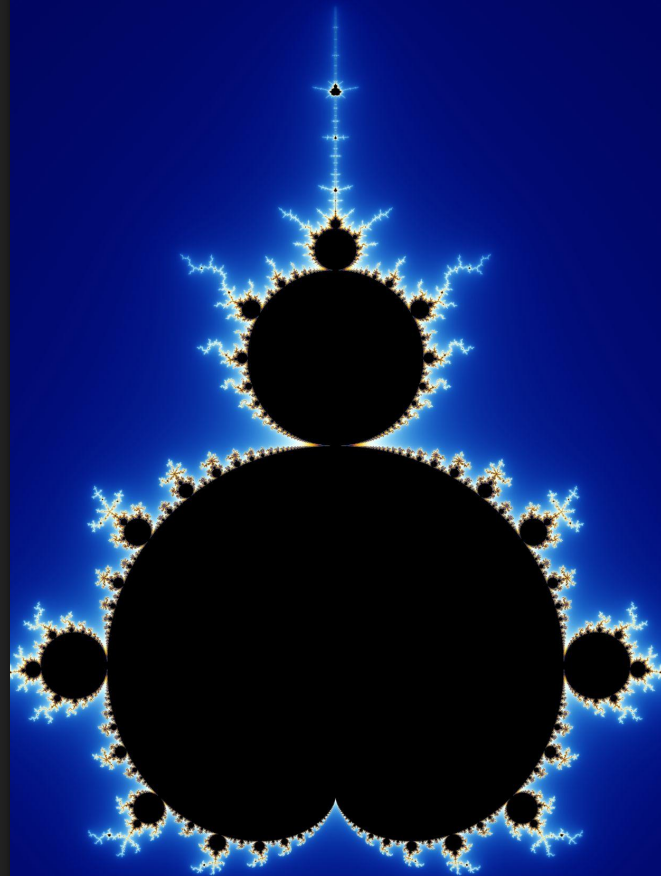
(C) Complex or Imaginary Numbers

- (R) Real Numbers
 1. (Q) Rational Numbers
 - (Z) Integers
 - (W) Whole Numbers
 - (N) Natural Numbers
 2. (Q') Irrational Numbers



Recursive Power (Mandelbrot Set)

1. Points plotted on complex plane to form fractal
2. Quadratic recurrence equation: $z_{n+1} = z_n^2 + c$
3. Boundary of Mandelbrot set contains infinitely many copies of Mandelbrot set.
4. Mathematical beauty
5. What is base case here? If exists, how is it ∞ ?
6. Mandelbrot Fractal Zoom [Link1](#) and [Link2](#)



Recap

- About Math
- Problem Recursion solves?
- What is Recursion?
- Simplest recursive pattern?
- Recursion vs Iteration
- Why Recursion?
- Why not Recursion?
- Mandelbrot set?
- What is base case?
- Structure of Recursion?
- Call Stack & Call Frame?
- Big O of Recursion?
- Types of Recursion?
- Recursion in world?

If I have to rate different math concepts based on their simplicity & practical applicability, then I will give 9/10 to 'Recursion or recurrence relation'. 90 % of my math work involves this single principle.

Vitthal Jadhav (Mathematician)

References

1. Cracking the Coding Interview Book - [Link](#)
2. What on Earth is Recursion? - [Link](#)
3. Understanding Recursion - [Link](#)
4. Patterns in Nature - [Link](#)
5. Intricate Ceilings of historic Iran - [Link](#)
6. Discover Kolams - [Link](#)
7. Many more...

